Sharif University of Technology

Department of Electrical Engineering

Course:

Embeded RealTime Systems

Project:

Design and Implementation of Distributed Database System

Introduction

Submitted by:
Mohammadhossein Sabzalian
Department of Electrical Engineering

Instructor:
Dr. Gholampoor

Term: Spring 2026
Submission: Jul 10, 2026
Due Date: Jul 10, 2026

Contents

Pre-Implementation Research Report

This section covers the foundational technical concepts required prior to beginning the design and implementation phases of the system.

Mongoose Library in C/C++

The **Mongoose Library** is an ultra-lightweight, event-driven networking library designed specifically for embedded systems and C/C++ applications. It provides a simple, non-blocking API that abstracts low-level socket programming into event-handler workflows.

> **Core Mechanics of Mongoose:** Mongoose operates via a central event manager loop (**mg_mgr_poll**). It handles network events such as connections, data arrival, and disconnections, invoking a user-defined callback function to process incoming buffers.

Applications and HTTP Server Implementation

Mongoose is widely used in resource-constrained environments like IoT devices, embedded Linux units, and performance-critical systems where pulling in large frameworks like Boost.Asio or Civetweb is impractical. It allows developers to turn an application into a fully functional **HTTP Server** with minimal code overhead.

To implement an HTTP server and an API endpoint, a network manager is initialized, a listener socket is bound, and the event loop is polled continuously. Incoming HTTP requests trigger an 'MG-EV-HTTP-MSG' event, allowing the server to parse URLs, read headers, and return JSON responses.

Listing 1: Minimal Concept of an HTTP API Endpoint with Mongoose (C/C++)

```c
#include "mongoose.h"

static void fn(struct mg_connection *c, int ev, void *ev_data) {
  if (ev == MG_EV_HTTP_MSG) {
    struct mg_http_message *hm = (struct mg_http_message *) ev_data;
    if (mg_match_uri(hm, "/api/status")) {
      mg_http_reply(c, 200, "Content-Type: application/json\r\n",
                    "{\"status\": \"active\", \"code\": 200}");
    } else {
      mg_http_reply(c, 404, "", "Not Found\n");
    }
  }
}
int main(void) {
  struct mg_mgr mgr;
  mg_mgr_init(&mgr);
  mg_http_listen(&mgr, "http://0.0.0.0:8000", fn, NULL);
  for (;;) mg_mgr_poll(&mgr, 1000);
  mg_mgr_free(&mgr);
  return 0;
}
```

Distributed Databases

A **Distributed Database** is a collection of multiple, logically interrelated databases spread over a computer network. Instead of relying on a single centralized machine, data storage and processing are distributed across multiple physical nodes to ensure scalability, fault tolerance, and high availability.

Common Architectural Paradigms

Depending on the consistency and performance trade-offs, various distributed database systems have been developed:

Key Distributed Database Systems

- **Apache Cassandra:** A highly scalable, peer-to-peer distributed NoSQL database. It utilizes a leaderless architecture with a distributed hash ring, making it exceptionally suited for **high-write throughput** applications like telemetry and real-time logging.
- **CockroachDB / Google Spanner:** Distributed SQL databases that maintain strict ACID compliance globally. They utilize the Raft consensus protocol and synchronized atomic clocks to support distributed transactions across multiple geographical regions.
- **MongoDB (Sharded Clusters):** A document-oriented NoSQL database that distributes data across clusters using a sharding key. It is highly optimized for rapid application development involving horizontal scaling of unstructured or semi-structured data.

Application Programming Interfaces (API)

An **API** acts as an intermediary software contract that allows two separate applications to communicate, exchange data, and share functionality without needing to understand each other's underlying internal source code.

Mechanisms of Inter-Service Communication

Modern software applications are increasingly built using a decentralized microservices paradigm. APIs act as the bridges between these distinct services through formal structural rules:

- **Encapsulation and Security:** An API exposes only designated entry points while hiding backend complexities and keeping database tables isolated from direct external user queries.
- **Standardized Formats:** By utilizing protocols like REST (HTTP methods with JSON payload formatting) or gRPC (Protocol Buffers over HTTP/2), services written in completely different languages (e.g., a C++ embedded core communicating with a Python data science backend) can perfectly deserialize data.

API Communication Flow: Client App sends Request → API Endpoint Router → Authentication/Validation Check → Backend Business Logic Execution → JSON Data Serialized → Formatted Response Returned to Client.

Local Area Network (LAN)

A **LAN** is a localized communication network that connects computers, microcontrollers, and secondary network infrastructure within a distinct, limited geographic area—such as an engineering lab, office building, or manufacturing plant.

Node Communication inside a LAN

Nodes within a local network establish peer connections through a combined software and hardware stack:

- **Physical Layer Addressing:** Every hardware adapter possesses a static media access control (**MAC address**) used to direct packets across localized network switches at Data Link Layer 2.
- **Network Layer Addressing (IP):** Nodes are assigned dynamic or static IP addresses within private Subnet blocks (e.g., '192.168.1.0/24').
- **Address Resolution Protocol (ARP):** Before an IP packet can leave a node, the device transmits an ARP broadcast to map the destination IP to its physical MAC address.

Broadcast Domains Excessive unmanaged node traffic within a single LAN segment can create broadcast storms, reducing total available bandwidth and degrading real-time system synchronization.

Distributed Systems

A **Distributed System** consists of an interconnected collection of independent autonomous computing elements that appears to its end-users as a single, unified, coherent system.

Core Benefits and Advantages

Building architectures across a distributed topology yields critical runtime advantages:

- **Horizontal Scalability:** Rather than performing expensive vertical upgrades on a single machine, extra computing resources can be added to the infrastructure seamlessly.
- **High Fault Tolerance:** The system avoids having a single point of failure; if one node crashes, consensus algorithms redistribute tasks to operational cluster replicas.

Design Challenges

Designing these environments introduces complex trade-offs summarized by the fundamental **CAP Theorem**:

$$\text{Consistency} + \text{Availability} + \text{Partition Tolerance} \rightarrow \text{Pick Two}$$

Question Engineering Challenges in Distributed Synchronization [Jump to Answer]

How do we ensure that separate physical nodes agree on the state of a system when network links fail? *Guidance:* Consider the **norm of the difference** between the state clocks of independent nodes and the latency delays introduced by consensus round-trips over physical network paths.

Answer to Architectural Mitigations [Back to Question]

To manage state drift, developers must incorporate distributed consensus engines (such as Raft or Paxos), manage strict data serialization formats, and account for network partition realities where synchronization must gracefully degrade into eventual consistency models.